Holes in Elaboration for Multimodal Dependent Type Theory

What goes wrong when extending Unification to MTT

Zoe Stafford[1]

University of Oxford
Based on work I did at the University of Aarhus, supervised by Daniel Gratzer

2026-02-11

[1] zoe.stafford@st-annes.ox.ac.uk

Outline

Elaboration

Multimodal Dependent Type Theory

Problems

What is Elaboration?

- Autocomplete for types

What is Elaboration?

- Autocomplete for types *and terms!*

User writes

```
sum : List Nat -> Nat
sum = \xs => foldr (\x, y => x + y) 0 xs
```

Elaborator generates

```
sum : List Nat -> Nat
sum = \xs : List Nat =>
    foldr {elem = Nat, acc = Nat}
          (\x : Nat, y : Nat => x + y) 0 xs
```

Why is Elaboration?

- Type theorists write

$$\frac{\Gamma, (x : A) \vdash t : B}{\Gamma \vdash \lambda(x : A)t : (x : A) \to B}$$

- Users want to write

$$\lambda x.t$$

- So *elaborator* fills in the rest
- See also: implicit arguments

```
id 10 ⟹ id {a = Nat} 10
```

How is Elaboration?

Metavariables and holes

- *Metavariables* stand in for unknown terms.
- Replace missing terms with metavariables.

$$\lambda x.x + 2 \implies \lambda(x : ?\mathtt{a}).x + 2$$

How is Elaboration?

Metavariables and holes

- *Metavariables* stand in for unknown terms.
- Replace missing terms with metavariables.

$$\lambda x.x + 2 \implies \lambda(x : ?\mathtt{a}).x + 2$$

- But how do we solve metavariables?

How is Elaboration?

Unification

- Unification = equality checking + solving metavariables
- How do we check `Refl : 2 = 1 + 1`

How is Elaboration?

Unification

- Unification = equality checking + solving metavariables
- How do we check `Refl : 2 = 1 + 1`
- We need to check equality between types/terms

$$\Gamma \vdash A \stackrel{?}{=} B$$

- What about `?a` $\stackrel{?}{=}$ `Nat`

How is Elaboration?

Unification

- Unification = equality checking + solving metavariables
- How do we check `Refl : 2 = 1 + 1`
- We need to check equality between types/terms

$$\Gamma \vdash A \stackrel{?}{=} B$$

- What about `?a` $\stackrel{?}{=}$ `Nat`
- Set `?a` $:=$ `Nat`

How is Elaboration?

What is equal?

$$\frac{}{\Gamma \vdash (\lambda x.s)(t) \equiv s[t/x]} \qquad \frac{\Gamma \vdash s : (x : A) \to B}{\Gamma \vdash s \equiv (\lambda x.s(x))} \qquad \cdots$$

- Checking structual equality = easy
- Checking β/η equality = hard

How is Elaboration?

What is equal?

$$\frac{}{\Gamma \vdash (\lambda x.s)(t) \equiv s[t/x]} \qquad \frac{\Gamma \vdash s : (x : A) \to B}{\Gamma \vdash s \equiv (\lambda x.s(x))} \qquad \cdots$$

- Checking structual equality = easy
- Checking β/η equality = hard

$$(\lambda x.?\mathtt{a})(10) \equiv ?\mathtt{a}[10/x] \not\equiv ?\mathtt{a}$$

How is Elaboration?

Solving metavariables

$$\Gamma \vdash ?\mathtt{a}[t_1/x_1, \ldots, t_n/x_n] \stackrel{?}{=} s$$

- $\gamma := (t_1/x_1, \ldots, t_n/x_n)$
- Want *unique* δ with $\gamma \circ \delta = \mathrm{id}$

$$?\mathtt{a} \equiv ?\mathtt{a}[\gamma \circ \delta] \equiv ?\mathtt{a}[\gamma][\delta] \equiv s[\delta]$$

- Pattern conditions:
  1. Each t_i is a distinct bound variable
  2. Each variable in s occurs in $t_1, \ldots, t_n$
  3. $?\mathtt{a}$ doesn't occur in s

MTT

Multimodal Dependent Type Theory

- MTT extends DTT with *modal operators*
- Modalities defined by 2-category $\mathcal{M}$
- 1-morphism $\mu : m \to n$ defines a modal operator $\langle \mu \mid - \rangle$
- 2-morphism $\alpha : \mu \Rightarrow \nu$ defines an operation $\langle \mu \mid - \rangle \to \langle \nu \mid - \rangle$

MTT

Example - Guarded recursion

- Modes: s = static data, t = temporal data
- $\blacktriangleright A = \langle \ell \mid A \rangle$ - values of type A that aren't available yet
- $\Delta A = \langle \delta \mid A \rangle$ - treat static data A as temporal data
- Axiom **löb** : $(\blacktriangleright A \to A) \to A$

$$\begin{aligned}
\mathrm{Stream}(A) &= \mathbf{löb}(\lambda S.\, \Delta A \times S) \\
\mathrm{head} &: \mathrm{Stream}(A) \to \Delta A \\
\mathrm{tail} &: \mathrm{Stream}(A) \to \mathrm{Stream}(A) \\
\mathrm{cons} &: \Delta A \to \mathrm{Stream}(A) \to \mathrm{Stream}(A)
\end{aligned}$$

MTT

Variables

- Variables annotated with modalities: $x :_\mu A$
- Contexts contain locks: $\{\mu\}$
- Variables need a 2-cell to be usable:

$$\frac{\alpha : \mu \Rightarrow \nu}{\Gamma.(x :_\mu A).\{\nu\} \vdash x^\alpha : A[\mathsf{key}^A]}$$

- A is in context $\Gamma.\{\mu\}$
- But we need it in $\Gamma.\{\nu\}$

MTT

Substitutions

- $\mathsf{sb}(\Delta, \Gamma) =$ substitutions from Δ to Γ
- $\delta \in \mathsf{sb}(\Delta, \Gamma)$ maps terms in Γ to terms in Δ

MTT and non-modal DTT:

$$\frac{}{\cdot \in \mathsf{sb}(\Delta, \cdot)} \qquad \frac{\delta \in \mathsf{sb}(\Delta, \Gamma) \qquad \Delta \vdash t : A[\delta]}{\delta.t \in \mathsf{sb}(\Delta, \Gamma.A)}$$

MTT only:

$$\frac{\mu : m \to n \qquad \delta \in \mathsf{sb}(\Delta, \Gamma)}{\delta.\{\mu\} \in \mathsf{sb}(\Delta.\{\mu\}, \Gamma.\{\mu\})} \qquad \frac{\alpha : \mu \Rightarrow \nu}{\mathsf{key}^{\alpha} \in \mathsf{sb}(\Gamma.\{\nu\}, \Gamma.\{\mu\})}$$

What goes wrong

- Normal pattern conditions aren't sufficient
- MTT substitutions are complicated

- In `mitten`, mode theory $\mathcal{M}$ is preorder-enriched
  - At most one operation $\mu \Rightarrow \nu$

Pattern conditions

$$\Gamma \vdash \texttt{?a}[\gamma] \stackrel{?}{=} s$$

- Original pattern conditions aren't sufficient

$$\begin{aligned} \text{unlater} &: (x :_\ell A) \to A \\ \text{unlater}(x) &= \texttt{?a} \end{aligned}$$

$$(x : A) \vdash \texttt{?a}[x^{1 \leq \ell}/x] \stackrel{?}{=} x$$

- But $\text{unlater}(x) = x$ isn't well-typed
- Want $(x :_\ell A), \{1\} \vdash x : A$ but $\ell \not\leq 1$

Pattern conditions

$$f : (x :_\ell A) \to \texttt{Either}\, A\, (\blacktriangleright A)$$
$$f(x) = \texttt{?b}$$

$$\text{Either} \quad (x : A) \vdash \texttt{?b}[x^{1 \leq \ell}/x] \stackrel{?}{=} \texttt{Left}(x) \tag{1}$$

$$\text{Or} \quad (x : A) \vdash \texttt{?b}[x^{1 \leq \ell}/x] \stackrel{?}{=} \texttt{Right}(\texttt{later}(x^{1 \leq \ell})) \tag{2}$$

- (1) - badly-typed - $f(x) = \texttt{Left}(x)$
- (2) - well-typed - $f(x) = \texttt{Right}(\texttt{later}(x))$
- Patten condition should depend on s

$$\Gamma \vdash \texttt{?a}[\gamma] \stackrel{?}{=} s$$

Pattern conditions

- Possible pattern condition: each variable is accessible in the solution

$$\Gamma \vdash \texttt{?a}[\gamma] \stackrel{?}{=} \texttt{Just}(\texttt{?b}[\delta])$$

- Need to check variables in δ
- Likely possible, but not in `mitten`

Normalisation by Evaluation (NbE)

- *environment* - map from variables to values
- $\mathsf{eval} : \mathsf{Term}(\Gamma) \times \mathsf{Env}(\Gamma) \to \mathsf{Value}$
- Avoids applying substitutions by updating the environment
- $\mathsf{eval}(\texttt{?a}, \sigma) = \texttt{?a}[\ldots]$

- Option 1: abandon NbE
- Option 2: pretend delayed substitutions are lists of terms

Pattern conditions

- Pattern condition: each variable is accessible in the solution

$$\Gamma \vdash \texttt{?a}[\gamma] \stackrel{?}{=} \texttt{Just}(\texttt{?b}[\delta])$$

- Solution: delay checking variables in δ until ?b is solved

Inverting substitutions

- Given substitution $\gamma \in \mathsf{sb}(\Gamma, \Delta)$ satisfies pattern conditions
- Compute action on variables:

$$v_\gamma : \mathsf{vars}(\Delta) \to \mathsf{vars}(\Gamma)$$
$$v_\gamma(x) = x[\gamma]$$

- Invert: $v_\gamma^{-1} : \mathsf{vars}(\Gamma) \rightharpoonup \mathsf{vars}(\Delta)$
- Create substitution γ^{-1}

Inverting substitutions

- Given substitution $\gamma \in \mathrm{sb}(\Gamma, \Delta)$ satisfies pattern conditions
- Compute action on variables:

$$v_\gamma : \mathrm{vars}(\Delta) \to \mathrm{vars}(\Gamma)$$
$$v_\gamma(x) = x[\gamma]$$

- Invert: $v_\gamma^{-1} : \mathrm{vars}(\Gamma) \rightharpoonup \mathrm{vars}(\Delta)$
- Create substitution γ^{-1}
- Ignore types to compute v_γ

Inverting substitutions

- Given substitution $\gamma \in \mathrm{sb}(\Gamma, \Delta)$ satisfies pattern conditions
- Compute action on variables:

$$v_\gamma : \mathrm{vars}(\Delta) \to \mathrm{vars}(\Gamma)$$
$$v_\gamma(x) = x[\gamma]$$

- Invert: $v_\gamma^{-1} : \mathrm{vars}(\Gamma) \rightharpoonup \mathrm{vars}(\Delta)$
- Create substitution γ^{-1}

- Ignore types to compute v_γ
- Don't compute γ^{-1}, just apply v_γ^{-1}
  - Doesn't work if solution contains substitutions

Inverting substitutions

- Given substitution $\gamma \in \mathrm{sb}(\Gamma, \Delta)$ satisfies pattern conditions
- Compute action on variables:

$$v_\gamma : \mathrm{vars}(\Delta) \to \mathrm{vars}(\Gamma)$$
$$v_\gamma(x) = x[\gamma]$$

- Invert: $v_\gamma^{-1} : \mathrm{vars}(\Gamma) \rightharpoonup \mathrm{vars}(\Delta)$
- Create substitution γ^{-1}

- Ignore types to compute v_γ
- Don't compute γ^{-1}, just apply v_γ^{-1}
  - Doesn't work if solution contains substitutions
- Pretend delayed substitutions are lists of terms

Conclusion

- Elaboration = autocomplete for types
- Solving metavariables needs:
  - Pattern conditions
  - Nice substitutions
- MTT breaks both of those
- `mitten` pretends substitutions are nice